

Experiment 7
Gyroscopic and Accelerometer (MPU6050)

Name: Ashutosh Bajpayee
CDSID: abajpaye



Table of Contents

Sl. No.	Topic	Page No.
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	4
5	Working Principle	4
6	Application Code	5-6
7	Output Image	7
8	Hardware Image	7
9	Appendix	8

Introduction

The MPU6050 is a 6-axis motion tracking sensor that integrates a 3-axis accelerometer and a 3-axis gyroscope. It communicates with microcontrollers through the I2C protocol. In this experiment, the sensor is interfaced with the Raspberry Pi Pico to obtain motion data such as acceleration (in g) and angular velocity (in degrees per second), which is then displayed on the serial console.

Scope

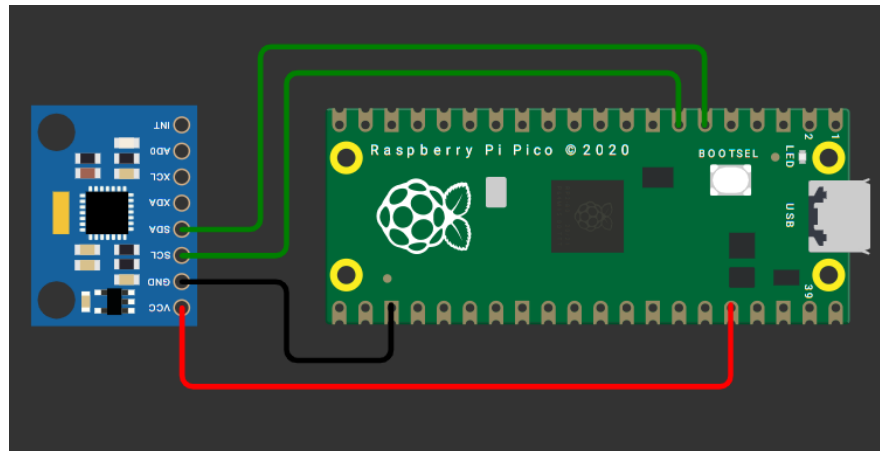
The scope of this experiment is to understand the I2C communication protocol and to interface the MPU6050 sensor with the Raspberry Pi Pico. It involves reading raw data from the sensor registers, converting these values into meaningful physical units, and displaying real-time motion data on the serial console.

Prerequisite

Hardware: Raspberry Pi Pico microcontroller, USB connector, breadboard, and MPU6050 sensor module.

Software: Visual Studio Code, PuTTY serial console, Pico SDK, CMake, and GCC toolchain.

Connection Diagram



Working Principle

- The MPU6050 communicates with the Raspberry Pi Pico using the I2C protocol, where the Pico acts as the master and the sensor as the slave.
- The Pico initializes I2C and wakes up the sensor by writing 0x00 to the power management register (0x6B).
- The sensor stores data in registers starting from 0x3B, and the Pico reads 14 bytes including accelerometer, temperature, and gyroscope data.
- The raw values are converted into physical units by dividing accelerometer data by 16384 (g) and gyroscope data by 131 (deg/s).
- The processed values are displayed on the serial console for real-time monitoring.

Application code

```
C Assignment_7.c > main()
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3  #include "hardware/i2c.h"
4
5  #define I2C_PORT i2c0
6  #define SDA_PIN 4
7  #define SCL_PIN 5
8
9  #define MPU_ADDR 0x68
10
11 void mpu_write(uint8_t reg, uint8_t data) {
12     uint8_t buf[2];
13     buf[0] = reg;
14     buf[1] = data;
15     i2c_write_blocking(I2C_PORT, MPU_ADDR, buf, 2, false);
16 }
17
18 void mpu_read(uint8_t reg, uint8_t *buffer, uint8_t length) {
19     i2c_write_blocking(I2C_PORT, MPU_ADDR, &reg, 1, true);
20     i2c_read_blocking(I2C_PORT, MPU_ADDR, buffer, length, false);
21 }
22
23 int16_t read_word(uint8_t reg) {
24     uint8_t data[2];
25     mpu_read(reg, data, 2);
26     return (int16_t)((data[0] << 8) | data[1]);
27 }
```

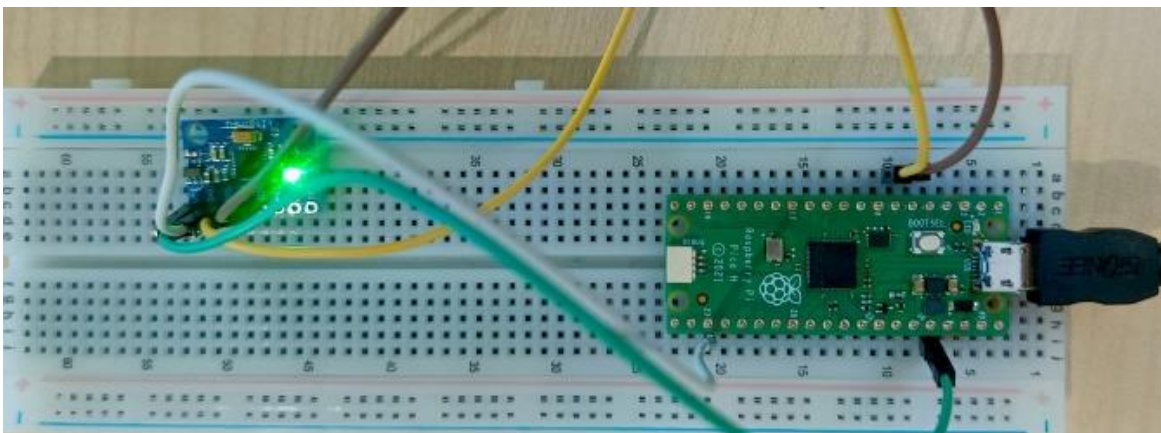
C Assignment_7.c > main()

```
29  int main() {
30      stdio_init_all();
31
32      i2c_init(I2C_PORT, 400 * 1000);
33      gpio_set_function(SDA_PIN, GPIO_FUNC_I2C);
34      gpio_set_function(SCL_PIN, GPIO_FUNC_I2C);
35      gpio_pull_up(SDA_PIN);
36      gpio_pull_up(SCL_PIN);
37      sleep_ms(2000);
38      printf("MPU6050 STARTED\n");
39
40      mpu_write(0x6B, 0x00);
41      while (1) {
42          uint8_t buffer[14];
43
44          i2c_write_blocking(I2C_PORT, MPU_ADDR, (uint8_t[]){0x3B}, 1, true);
45          int ret = i2c_read_blocking(I2C_PORT, MPU_ADDR, buffer, 14, false);
46          if (ret < 0) {
47              printf("READ ERROR\n");
48              sleep_ms(1000);
49              continue;
50          }
51          int16_t acc_x = (buffer[0] << 8) | buffer[1];
52          int16_t acc_y = (buffer[2] << 8) | buffer[3];
53          int16_t acc_z = (buffer[4] << 8) | buffer[5];
54
55          int16_t gyro_x = (buffer[8] << 8) | buffer[9];
56          int16_t gyro_y = (buffer[10] << 8) | buffer[11];
57          int16_t gyro_z = (buffer[12] << 8) | buffer[13];
58
59          float ax = acc_x / 16384.0;
60          float ay = acc_y / 16384.0;
61          float az = acc_z / 16384.0;
62
63          float gx = gyro_x / 131.0;
64          float gy = gyro_y / 131.0;
65          float gz = gyro_z / 131.0;
66
67          printf("Accelerometer (g): X=%.2f Y=%.2f Z=%.2f\n", ax, ay, az);
68          printf("Gyroscope (deg/s): X=%.2f Y=%.2f Z=%.2f\n", gx, gy, gz);
69
70          printf("-----\n");
71
72          sleep_ms(500);
73      }
```

Output Image

```
COM9 - PuTTY
Gyroscope (deg/s): X=29.82 Y=65.98 Z=21.14
-----
Accelerometer (g): X=-0.64 Y=0.39 Z=-0.04
Gyroscope (deg/s): X=-17.28 Y=-20.60 Z=-44.60
-----
Accelerometer (g): X=0.71 Y=-0.12 Z=0.73
Gyroscope (deg/s): X=-101.34 Y=-250.14 Z=-21.56
-----
Accelerometer (g): X=0.46 Y=-0.17 Z=-1.02
Gyroscope (deg/s): X=-34.21 Y=-60.66 Z=7.20
-----
Accelerometer (g): X=0.25 Y=-0.16 Z=-1.01
Gyroscope (deg/s): X=11.35 Y=9.30 Z=-16.21
-----
Accelerometer (g): X=0.30 Y=-0.19 Z=-1.00
Gyroscope (deg/s): X=-11.87 Y=15.51 Z=2.01
-----
Accelerometer (g): X=0.98 Y=-0.39 Z=0.15
Gyroscope (deg/s): X=77.49 Y=162.66 Z=-4.99
-----
Accelerometer (g): X=-0.31 Y=0.11 Z=1.01
Gyroscope (deg/s): X=6.82 Y=136.56 Z=-15.95
-----
Accelerometer (g): X=0.27 Y=0.03 Z=0.98
Gyroscope (deg/s): X=20.57 Y=-4.05 Z=-2.14
-----
Accelerometer (g): X=0.25 Y=0.10 Z=1.13
Gyroscope (deg/s): X=8.27 Y=42.47 Z=11.95
```

Hardware Image



Appendix

- The experiment uses a Raspberry Pi Pico, MPU6050 sensor module, breadboard, and connecting wires.
- I2C protocol is used for communication between the microcontroller and the sensor.
- The MPU6050 provides accelerometer and gyroscope data through its registers.
- Raw sensor values are converted into physical units for better interpretation.
- The output is displayed on the serial console, confirming correct interfacing and operation.